Kubernetes + Helm + Dorarion

> Beta 数据库说明（2026-10-10）：当前 core 使用两个合并后的迁移文件。替换旧 Beta 数据库需要新的空数据库或明确删除旧数据后重新安装。不要只删除 SQLx 记录。请从当前源码构建新镜像，并参考 README 的最新政策；以前下载的 PDF 可能包含旧流程。

请使用根路径的独立 HTTPS 主机名，不支持子路径部署。仅一次性明文 PostgreSQL 示例使用下面的 Helm 参数明确允许不加密的数据库连接。生产外部数据库必须使用 sslmode=verify-full 和认可的 CA。

本指南与当前源代码检出目录匹配。安装前请构建并验证匹配的 core 镜像；旧的已发布 Chart 和镜像不包含这些修复。需要 PostgreSQL 16 或更高版本。

简易集群安装指南

> **目标：** 使用当前检出的 Helm Chart 安装 Dorarion，连接外部 PostgreSQL 数据库，创建所需的 Kubernetes Secret 和第一个管理员账户，然后验证应用。

- 请在已配置为访问目标 Kubernetes 集群的终端中运行所有命令。
- 请在更新后的 `dorarion-helm` 检出目录根目录中运行。
- Dorarion 需要 Kubernetes 1.28 或更高版本以及 Helm 3。
- 此 Chart 不安装 PostgreSQL；请先准备数据库和备份。
- 另有 PDF 版本。

1. 检查 Kubernetes 和 Helm

在创建任何资源之前，确认 `kubectl` 指向正确的集群：

```
kubectl config current-context
kubectl cluster-info
kubectl version
helm version
```

如果 context 指向错误的集群，请不要继续。Kubernetes 服务端版本必须至少为 1.28。

2. 检查本地 Chart 和匹配的镜像

查看当前检出目录，并选择从匹配的 core 源代码构建的已验证镜像：

```
helm show chart .
```

```
read -r -p "Verified matching image repository: " DORARION_IMAGE_REPOSITORY
read -r -p "Verified image digest (sha256:...): " DORARION_IMAGE_DIGEST
```

Chart 元数据如下：

- Chart：`dorarion`
- Chart 版本：`0.0.3`
- 应用版本：`0.0.3`
- 应用镜像：已验证的仓库和不可变 digest。

3. 准备 PostgreSQL

创建一个空的 PostgreSQL 数据库和 Dorarion 专用用户。该用户必须能够连接， 创建和修改表、索引及约束，并运行 Dorarion 数据库迁移。

请根据数据库服务商的要求在连接 URL 中加入 TLS 参数。典型 URL 格式如下：

```
postgres://USER:PASSWORD@HOST:5432/DB_NAME?sslmode=verify-full
```

每次升级 Dorarion 前都要备份数据库，并在隔离环境中测试恢复。Helm Chart 不会管理或备份 PostgreSQL。

可选：运行简单的 PostgreSQL Pod

如果还没有 PostgreSQL，可以使用本仓库提供的单节点 PostgreSQL 示例。它会创建一个 10 Gi 持久卷声明、 一个 PostgreSQL Deployment 和一个内部 ClusterIP Service。

> **此示例仅用于学习和小型测试安装。** 它不具备高可用性，不配置 TLS， 也不创建备份。生产环境请使用托管数据库或持续维护的 PostgreSQL Operator。

创建 namespace 和随机 PostgreSQL 密码：

```
kubectl create namespace dorarion --dry-run=client -o yaml | kubectl apply -f -

POSTGRES_PASSWORD="$(openssl rand -hex 32)"

kubectl -n dorarion create secret generic dorarion-postgresql \
  --from-literal=POSTGRES_DB=dorarion \
  --from-literal=POSTGRES_USER=dorarion \
  --from-literal=POSTGRES_PASSWORD="$POSTGRES_PASSWORD"
```

部署当前检出目录的 PostgreSQL 示例，并等待其就绪：

```
kubectl apply -f examples/postgresql.yaml

kubectl -n dorarion rollout status deployment/dorarion-postgresql \
```

```
  --timeout=5m
kubectl -n dorarion get pod,service,persistentvolumeclaim \
  -l app.kubernetes.io/name=dorarion-postgresql
```

如果创建 PostgreSQL 后打开了新终端，可在不显示密码的情况下将其恢复到变量：

```
POSTGRES_PASSWORD="$(kubectl -n dorarion get secret dorarion-postgresql \
  -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 --decode)"
```

使用此示例时，设置下一节所需的数据库 URL：

```
DATABASE_URL="postgres://dorarion:${POSTGRES_PASSWORD}@dorarion-
postgresql.dorarion.svc.cluster.local:5432/dorarion"
DORARION_DEMO_HELM_ARGS=(--set extraEnv[0].name=DORARION_ALLOW_INSECURE_DATABASE --set-
string extraEnv[0].value=true)
```

4. 创建 namespace 和 Secret

数据库迁移作为 pre-install hook 运行，因此必须在 `helm install` 之前创建 Secret。

创建 namespace：

```
kubectl create namespace dorarion --dry-run=client -o yaml | kubectl apply -f -
```

生成应用密钥并创建 Kubernetes Secret。如果使用了 PostgreSQL 示例， `DATABASE_URL` 已经设置。使用外部数据库时，请先替换并运行：

```
DATABASE_URL='postgres://USER:PASSWORD@HOST:5432/DB_NAME?sslmode=verify-full'
DORARION_DEMO_HELM_ARGS=()
```

然后创建应用 Secret：

```
: "${DATABASE_URL:?Set DATABASE_URL before continuing}"
JWT_SECRET="$(openssl rand -hex 48)"
SECRET_ENCRYPTION_KEY="$(openssl rand -base64 32)"
DORARION_BOOTSTRAP_PASSWORD="$(openssl rand -hex 24)"

kubectl -n dorarion create secret generic dorarion-secrets \
  --from-literal=DATABASE_URL="$DATABASE_URL" \
  --from-literal=JWT_SECRET="$JWT_SECRET" \
  --from-literal=SECRET_ENCRYPTION_KEY="$SECRET_ENCRYPTION_KEY" \
  --from-literal=DORARION_BOOTSTRAP_PASSWORD="$DORARION_BOOTSTRAP_PASSWORD"

unset DATABASE_URL POSTGRES_PASSWORD JWT_SECRET SECRET_ENCRYPTION_KEY
DORARION_BOOTSTRAP_PASSWORD
```

> **请妥善保护这些值。** 不要提交到 Git、写入 Helm values，或发布到聊天、 截图和工单中。请将 `SECRET_ENCRYPTION_KEY` 另外备份到密码管理器或密钥 管理系统。丢失此密钥后，直接存储在 Dorarion 中的集成凭据将无法恢复。

仅检查 Secret 的键名，不显示其值：

```
kubectl -n dorarion describe secret dorarion-secrets
```

必须包含 `DATABASE_URL`、`JWT_SECRET`、`SECRET_ENCRYPTION_KEY` 和 `DORARION_BOOTSTRAP_PASSWORD`。

5. 安装 Dorarion 并创建第一个管理员

选择用户和 CI Runner 实际可访问的 HTTPS 地址。仅在首次安装时启用管理员 bootstrap：

```
helm upgrade --install dorarion \
  . \
  "${DORARION_DEMO_HELM_ARGS[@]}" \
  --namespace dorarion \
  --set-string image.repository="$DORARION_IMAGE_REPOSITORY" \
  --set-string image.digest="$DORARION_IMAGE_DIGEST" \
  --set publicUrl=https://dorarion.example.com \
  --set bootstrapAdmin.enabled=true \
  --set bootstrapAdmin.username=admin \
  --set bootstrapAdmin.email=admin@example.com \
  --wait \
  --timeout 15m
```

该地址必须可被用户、GitLab Runner 以及向 Dorarion 发送报告或 webhook 的 系统访问。生产环境应使用 HTTPS。

安装按以下顺序进行：

1. 运行阻塞式数据库 migration Job；
2. 创建 Dorarion Deployment 和 Service；
3. 等待应用就绪；
4. 运行首个管理员 bootstrap Job。

6. 保存管理员密码

bootstrap 从预先创建的 Secret 读取 DORARION_BOOTSTRAP_PASSWORD（至少 16 个字符），不会输出密码。在清除 shell 变量前保存到密码管理器。登录时从管理器或 Secret 获取密码。验证后可删除已完成的 Job。已有安装不要再次启用 bootstrap。

```
kubectl -n dorarion delete job dorarion-bootstrap-admin
```

7. 检查安装

检查 Helm Release 和 Kubernetes 工作负载：

```
helm status dorarion --namespace dorarion
kubectl -n dorarion get deployment,pods,service,jobs
kubectl -n dorarion rollout status deployment/dorarion
```

如果迁移失败，请检查保留的 Job 和日志：

```
kubectl -n dorarion get jobs
kubectl -n dorarion logs job/dorarion-migrate
```

常见原因包括 `DATABASE_URL` 错误、到 PostgreSQL 的网络访问被阻止、Secret 键缺失或数据库权限不足。

8. 在本机打开 Dorarion

无需 Ingress 即可进行快速检查，只需转发 ClusterIP Service：

```
kubectl -n dorarion port-forward service/dorarion 3000:80
```

保持该终端打开，然后访问：

http://127.0.0.1:3000/login

使用 `admin` 或 `admin@example.com` 以及第 6 步保存的密码登录。在另一个终端 检查 readiness：

```
curl --fail http://127.0.0.1:3000/health/ready
```

9. 配置 Ingress

必须已经部署 Ingress Controller 并配置 DNS。创建不含密钥的本地 values 文件 `values.local.yaml`：

```
publicUrl: https://dorarion.example.com

ingress:
  enabled: true
  className: nginx
  hosts:
    - host: dorarion.example.com
      paths:
        - path: /
          pathType: Prefix
  tls:
    - secretName: dorarion-tls
```

```
      hosts:
        - dorarion.example.com
```

在不启用管理员 bootstrap 的情况下应用：

```
helm upgrade dorarion \
  . \
  "${DORARION_DEMO_HELM_ARGS[@]}" \
  --namespace dorarion \
  --reuse-values \
  --set bootstrapAdmin.enabled=false \
  --values values.local.yaml \
  --wait \
  --timeout 15m
```

请根据 Ingress Controller 或证书管理器的方式配置 TLS Secret。不要将 TLS 私钥写入 values 文件。

10. 升级 Dorarion

升级前请执行以下操作：

1. 备份 PostgreSQL 并验证备份能够恢复；
2. 备份准确的 `SECRET_ENCRYPTION_KEY` 及其密钥引用；
3. 阅读 Release Notes 和迁移兼容性策略；
4. 选择新的不可变 Chart 和应用版本。

保持管理员 bootstrap 禁用并升级：

```
helm upgrade dorarion \
  . \
  --version NEW_CHART_VERSION \
  --namespace dorarion \
  --reuse-values \
  --set bootstrapAdmin.enabled=false \
  --wait \
  --timeout 15m
```

pre-upgrade migration 会向前修改数据库。回滚 Helm Release 不会撤销数据库迁移。

11. 卸载范围

删除 Helm Release：

```
helm uninstall dorarion --namespace dorarion
```

这不会删除外部 PostgreSQL 数据库或手动创建的 `dorarion-secrets` Secret。 如果以后可能重新安装或恢复，请保留两者。只有在确定要删除 namespace 中所有 剩余资源时，才删除该 namespace。

如果部署了 PostgreSQL 示例，其资源也独立于 Helm Release。使用常见 `Delete` reclaim policy 的 StorageClass 时，删除持久卷声明也会删除数据库：

```
kubectl -n dorarion delete deployment,service dorarion-postgresql
kubectl -n dorarion delete persistentvolumeclaim dorarion-postgresql
kubectl -n dorarion delete secret dorarion-postgresql
```

除非确定要销毁示例数据库并已取得所需备份，否则不要执行持久卷声明删除命令。

快速回顾

- 已检查 Kubernetes context 和版本。
- 已检查 Docker Hub 上发布的 OCI Chart。
- 已准备外部 PostgreSQL 数据库和备份。
- 或者仅为测试部署了单节点 PostgreSQL 示例。
- 已创建 namespace 和必需的 Kubernetes Secret。
- 仅在首次安装时启用 bootstrap 并安装 Dorarion。
- 已在密码管理器保存预先创建的管理员密码；bootstrap 日志不包含密码。
- 已验证 Deployment、readiness 和浏览器访问。
- 后续升级保持 bootstrap 禁用。

官方参考资料

- Dorarion Helm 源代码
- Docker Hub 上的 Dorarion Chart
- 安装 Helm
- Helm OCI Registry
- 安装 kubectl

安装与升级 — reviewed core

这些设置针对已审查的 1.0 之前 core 源码。请选择由该源码构建的镜像；现有 v0.0.3 标签不能证明包含本次修复。

1.0 之前版本：请使用所选镜像附带的 API 契约。已审查源码支持 CI/CD、运维任务、事故及每日/每周活动报告。

- 一次生成 `JWT_SECRET`（至少 32 字节）和 `SECRET_ENCRYPTION_KEY`（32 个随机字节的 Base64），不要提交到 Git。
- API 连接池默认 15；每个副本还需预留 6 个 worker 连接及临时迁移连接。
- Compose 仅显式信任内部 `postgres` 主机；外部 PostgreSQL 需要 `sslmode=verify-full` 和可信 CA。
- 外部凭据使用 `env://DORARION_EXT_<NAME>`；core 秘密环境变量始终被拒绝。

- Argo CD 使用独立轮换的应用令牌，应用和集成都须启用。事故自动化使用限定仓库的 incident source。

`/health/live` 检查进程，`/health/ready` 检查 PostgreSQL。worker 状态单独通过需认证的 `/health/workers` 查看。Prometheus 使用 Secret 中 32–512 个 ASCII 字符的 `DORARION_METRICS_SCRAPE_TOKEN`，仅发送至 `/metrics`。接受转发的客户端地址前必须配置可信代理网络。

前向迁移前备份 PostgreSQL 和原始加密密钥。单独运行迁移 Job/服务，关闭服务器启动迁移。首次登录修改生成的管理员密码，升级后轮换 Argo 应用令牌。审计保留不能短于 365 天。每周报告覆盖发送前完整的本地 7 天；重试同一次手动发送应复用 Idempotency-Key。重新打开事故必须显式请求。演示端点已删除，未知请求字段被拒绝，请检查镜像内的 API 契约。上线前在自己的环境验证恢复、供应商消息和 readiness。

Helm: `database.connectionPoolSize=15`, `database.statementTimeoutMs=30000`, `database.trustedHosts=[]`, `security.secretEnvAllowlist=[]`, `security.trustedProxies=[]`, `security.reportRecipientDomains=[]`, `security.readmeImageOrigins=[]`. Optional monitoring credentials belong in the existing Secret; select their key with `security.metricsScrapeTokenSecretKey`. Hook Jobs receive the same policy and `extraEnvFrom` as the server.